

ExplainEat

System- und KI-Dokumentation

Projektdokumentation für die Schweizer KI Challenge 2026

Technischer Schwerpunkt

PyTorch-Modell, Feature-Vektor, Training, Inferenz und End-to-End-Integration

Dokumentationsprinzip: Diese Fassung beschreibt das System aus Sicht einer technischen Prüfung. Sie trennt nachweisbare Eigenleistung, externe Komponenten, aktuelle Funktionen, geplante Erweiterungen und noch offene Belege. Unverifizierte Angaben werden nicht als abgeschlossen dargestellt.

Stand: 26. Juli 2026

Projektstatus: Prototyp

Inhalt

1. Projektkontext, Ziel und Schwerpunkt
2. Abgleich mit den Anforderungen der Schweizer KI Challenge 2026
3. Eigenleistung, externe Komponenten und Unterstützungsnachweis
4. Systemarchitektur und Laufzeitablauf
5. Der eigene KI-Kern: PyTorch-Modell und Feature-Vektor
6. Training, Evaluation und Ausbau des Modells
7. Zwei Use Cases und ihr Nutzen

Für die Abgabe entscheidend: Die offizielle Challenge erwartet ausführbaren Code mit README, einen zweiminütigen Video-Pitch und eine kritische Reflexion. Zusätzlich muss Unterstützung durch Personen, Institutionen, bestehende Algorithmen, Datensätze oder Werkzeuge transparent ausgewiesen werden.

1. Projektkontext, Ziel und Schwerpunkt

1.1 Entwicklung für die Schweizer KI Challenge 2026

Ich habe ExplainEat für die Schweizer KI Challenge 2026 konzipiert und als funktionsfähigen Prototyp umgesetzt. Im Mittelpunkt steht nicht das visuelle Design der App, sondern die Frage, wie aus einem Foto, bestätigten Lebensmitteldaten und einem Nutzerprofil eine nachvollziehbare, profilbezogene Bewertung entsteht.

Das praktische Problem ist einfach zu beschreiben: Nährwertangaben sind zwar verfügbar, werden aber häufig isoliert betrachtet. Eine Zahl für Kalorien, Zucker oder Protein erklärt noch nicht, wie gut eine konkrete Mahlzeit zu einem Ziel, einer Portionsgrösse oder zu bestätigten Einschränkungen passt. ExplainEat führt diese Informationen in einem kontrollierten Ablauf zusammen und erklärt das Ergebnis in verständlicher Sprache.

Die Dokumentation richtet sich bewusst an zwei Zielgruppen. Nicht-Expertinnen und Nicht-Experten sollen den Nutzen und die Grenzen verstehen. Technische Prüferinnen und Prüfer sollen erkennen können, welche Komponenten ich selbst entwickelt habe, welche extern sind und welche Aussagen erst durch Code, Tests oder ein Screenrecording belegt werden müssen.

ExplainEat in drei Sätzen:

ExplainEat ist ein hybrides KI-System zur profilbezogenen Bewertung von Ernährung/Mahlzeiten. Der zentrale KI-Kern ist ein von mir entworfenes neuronales Netz, das ich mit dem PyTorch-Framework implementiert und auf rund 30'000 synthetisch erzeugten Trainingsbeispielen trainiert habe. Den Feature-Vektor, die Bewertungsziele, die Trainingslogik und die Integration in das Flask-Backend habe ich selbst definiert und mit VisualStudio/Claude Code entwickelt. Externe KI-Komponenten werden zusätzlich für die Bildanalyse und die sprachliche Erklärung eingesetzt und klar von meinem eigenen Bewertungsmodell abgegrenzt.

1.2 Bewusste Abgrenzung: KI vor Design

- Funktionales Frontend statt Designprojekt: Die Flutter-App dient als Eingabe-, Korrektur- und Demonstrationsoberfläche. Branding, Animationen und eine marktreife User Experience waren nicht der Schwerpunkt.
- Eigener Machine-Learning-Prozess: Der grösste technische Aufwand liegt in Datengenerierung, Modelltraining, Evaluation und Integration des Modells in den Laufzeitprozess.
- End-to-End-Integration: Das Modell ist in einen Ablauf aus Bildanalyse, Nutzerbestätigung, Datenanreicherung, Regelprüfung, Flask-API und Erklärungsschicht eingebunden.
- Prototyp statt Medizinprodukt: ExplainEat ist eine Lern- und Forschungsanwendung. Das System ersetzt keine Ernährungsberatung, Diagnose oder medizinische Behandlung.
- Nachvollziehbarkeit vor Inszenierung: Für die Bewertung sind reproduzierbarer Code, klare Metriken, Quellen, Unterstützungsnachweise und eine ehrliche Darstellung der Grenzen wichtiger als eine perfekte Oberfläche.

1.3 Was ich mit ExplainEat nachweisen möchte

- Ich kann ein reales Problem in strukturierte Eingaben, Merkmale und Zielwerte für maschinelles Lernen übersetzen.
- Ich kann synthetische Trainingsdaten erzeugen, deren Herkunft dokumentieren und ihre Aussagekraft kritisch begrenzen.
- Ich kann mit PyTorch ein eigenes neuronales Netz definieren, trainieren, speichern und während der Laufzeit verwenden.
- Ich kann externe KI-Komponenten sauber abgrenzen: Bildanalyse und Sprachmodell unterstützen den Ablauf, ersetzen aber nicht mein eigenes Bewertungsmodell.
- Ich kann Fehlerfortpflanzung, Unsicherheit, Verzerrungen, Datenschutz und fachliche Grenzen systematisch beschreiben.

1.4 Wichtigste technische Aussage

Die eigentliche technische Leistung besteht aus dem formulierten Lernproblem, dem Datenmodell, dem Feature-Vektor, der Modellarchitektur, dem Trainingsprozess, der Evaluation und der Integration in die Anwendung. Diese Unterscheidung ist für die Bewertung der Eigenleistung zentral.

2. Abgleich mit den Anforderungen der Schweizer KI Challenge 2026

2.1 Verlangte Bestandteile der Projektabgabe

Offizielle Anforderung	Umsetzung für ExplainEat	Prüfstatus
Ausführbarer Code mit kurzer Anleitung / README	Backend, Trainingsskript, Inferenzcode, Modellartefakt, Abhängigkeiten, Konfiguration und Beispielinput müssen als reproduzierbares Paket vorliegen. Falls die Ausführung auf einem fremden Computer nicht möglich ist, sollte ein Screenrecording den Ablauf belegen.	Dokumentiert; technisch zu prüfen. Erfüllt
2-minütiger Video-Pitch	Der Pitch zeigt Motivation, Ziel, verwendete KI-Methode, kurze Demo, Ergebnis, wichtigste Herausforderung und ehrliche Grenze.	Erfüllt
Kritische Reflexion	Zielerreichung, Eigenleistung, Daten, Methoden, Metriken, Fehlerquellen, Unsicherheit, Bias, Missbrauchsrisiken und Grenzen werden schriftlich reflektiert.	Erfüllt
Quellen und Unterstützung offenlegen	Frameworks, externe Modelle, öffentliche Daten, Coding-Unterstützung, Coaching und weitere Hilfen werden mit Art und betroffenem Projektteil aufgeführt.	Erfüllt

2.2 Bewertungskriterien – Prüfmatrix

Kriterium	Nachweis in der Dokumentation	Auditstatus
Eigenständigkeit	Eigener Problemzuschnitt, Bewertungslogik, Datengenerator, Feature-Vektor, PyTorch-Modell, Training und Integration. Externe Komponenten werden getrennt ausgewiesen.	Inhaltlich erfüllt
Funktionalität des Codes	Soll-Ablauf und Schnittstellen sind beschrieben. Der Nachweis erfolgt durch Ausführung auf einem fremden System oder durch ein vollständiges Screenrecording.	Zu prüfen durch Audit
Lesbarkeit des Codes	README, Modulstruktur, Konfiguration, Beispielinput, Fehlerbehandlung und Kommentare an nicht offensichtlichen Stellen.	Am Repository prüfen
Schwierigkeitsgrad und Aufwand	Datengenerierung, Preprocessing, neuronales Netz, REST-Integration, Bildanalyse, Regelwerk und Erklärungsschicht bilden einen anspruchsvollen End-to-End-Prototyp.	Plausibel; durch Code belegen
Originalität und Kreativität	Kombination aus visueller Erfassung, Nutzerkorrektur, profilbezogener ML-Bewertung und erklärender Ausgabe.	Inhaltlich erfüllt
Erkenntnisgewinn und Relevanz	Das Projekt zeigt, wie Datenqualität, Portionsunsicherheit, Profilkontext und Modellgrenzen das Ergebnis beeinflussen.	Inhaltlich erfüllt
Wissenschaftliches Arbeiten	Fehlerquellen, Baseline-Vergleich, Testtrennung, Metriken, Reproduzierbarkeit und Stand der Technik müssen nachvollziehbar dokumentiert sein.	Teilweise; Messwerte ergänzen
Selbstkritische Einschätzung	Synthetische Labels und die 98-Prozent-Aussage werden korrekt eingeordnet. Medizinische Grenzen werden ausdrücklich benannt.	Erfüllt

Zentrale Feststellung: Die grösste fachliche Stärke ist die transparente Trennung der Komponenten. Die grösste offene Frage ist, ob das trainierte Modell gegenüber der direkten Regelberechnung einen messbaren Nutzen liefert. Genau dieser Vergleich sollte in Code und Video gezeigt werden.

3. Eigenleistung, externe Komponenten und Unterstützungsnachweis

3.1 Meine eigene technische Arbeit

- Ich habe Problemstellung, Zielgruppe und fachliche Abgrenzung von ExplainEat definiert.
- Ich habe die Bewertungslogik und die Regeln für unterschiedliche Profile und Mahlzeiten entwickelt.
- Ich habe einen Generator für rund 30'000 synthetische Trainingsbeispiele konzipiert und eingesetzt.
- Ich habe die Eingabemerkmale und die feste Reihenfolge des Feature-Vektors festgelegt. Die finale Reihenfolge muss mit dem Code und dem Preprocessing-Artefakt übereinstimmen.
- Ich habe das neuronale Netz in PyTorch aufgebaut mit Code unterstützung, trainiert, validiert, gespeichert und in die Inferenz des Backends eingebunden.
- Ich habe das Flask-Backend als Orchestrierungsschicht zwischen App, Bildanalyse, Datenebene, Regelwerk, PyTorch-Modell und Erklärungskomponente umgesetzt.
- Ich habe Allergie-, Rezept- und Empfehlungslogik sowie die Zuordnung von Lebensmitteln zu Nährwerten aufgebaut.
- Ich habe eine funktionale Flutter-Oberfläche erstellt, ohne den Schwerpunkt auf visuelles Produktdesign zu legen.

3.2 Verwendete Frameworks, Modelle und Hilfsmittel

Komponente	Rolle im System	Meine Eigenleistung	Externer Anteil
PyTorch	Framework für Tensoren, neuronales Netz, Training und Inferenz.	Lernproblem, Feature-Auswahl, Datengenerator, Modellaufbau, Training, Tests und Integration.	Framework, Autograd, Standard-Layer, Optimizer und Daten-Utilities.
SAM2 / Roboflow	Bildbereiche segmentieren und – je nach eingesetztem Modell – Objekte oder Lebensmittelklassen erkennen.	Anbindung, Datenfluss, Konfidenzbehandlung, Fehlerfälle und Übergabe an die eigene Bewertung.	Vortrainierte Segmentierungsmodelle, Hosting- oder Inferenzfunktionen und gegebenenfalls externe Erkennungsmodelle.
Visual Studio (VS), Claude Code via PlugIN (VS Code)	Genrierung von Code	Promptstruktur, erlaubte Eingaben, Ausgaberegeln, Sicherheitsgrenzen und Systemintegration. Prüfen des Codes, Testing	Sprachmodell und dessen allgemeine Sprachfähigkeit. Claude berechnet nicht den verbindlichen Score.
Flask	REST-Backend und technische Orchestrierung.	API-Endpunkte, Validierung, Workflow, Fehlerbehandlung, Modellaufruf und Ergebnisobjekt.	Webframework und Standardfunktionen.
Flutter	Funktionale Eingabe-, Korrektur- und Ergebnisoberfläche.	Formulare, Fotoübergabe, API-Aufruf, Nutzerkorrektur und Ergebnisdarstellung.	App-Framework und UI-Komponenten.
Öffentliche Nährwertdaten	Referenzwerte für Lebensmittel und Rezepte.	Datenmodell, Zuordnung, Einheiten, Plausibilisierung und Verwendung im Feature Engineering.	Ursprüngliche Datenbestände. Konkrete Quellen und Lizenzen müssen vor Abgabe genannt werden.
KI-Unterstützung beim Programmieren	Unterstützung bei Boilerplate, Debugging, Refactoring und Formulierungen.	Architektur- und Fachentscheidungen, Prüfung der Vorschläge und Verantwortung für den finalen Code.	Generierte Vorschläge und sprachliche Unterstützung.

3.3 Unterstützungsnachweis vor der Abgabe

Person / Institution / Tool	Art der Unterstützung	Betroffener Projektteil	Status
ETH-Coaching: Frau Balmer	Fachliches Feedback, Coaching, Bereitstellung von Informationen und Unterlagen.	Generelle Einschätzung und Gewichtung zum Thema «eigenen KI».	Erfolgt
Dokumentationen zu Visual Studio, Claude Code, Flask und Flutter	Boilerplate, Fehlersuche, Refactoring und Formulierungshilfen	Code und Dokumentation	Dokumentiert
Roboflow / Meta SAM2	Bereitgestellte Bildanalyse-, Segmentierungs- oder Hosting-Technologie.	Bildanalyse, Workflow	Dokumentiert

4. Funktions- und Schnittstellenübersicht

ExplainEat besteht aus einem Offline-Teil für Datenaufbau und Training sowie einem Laufzeit-Teil für die Bewertung einer konkreten Aufgabe/Mahlzeit. Diese Trennung verhindert, dass Training und produktive Verarbeitung vermischt werden. Sie macht ausserdem sichtbar, an welcher Stelle externe Modelle verwendet werden und wo mein eigenes PyTorch-Modell arbeitet. In der Folge wird ExplainEat Schritt-für-Schritt erklärt.

ExplainEat – Funktions- und Schnittstellenübersicht

Prototyp / Proof of Concept

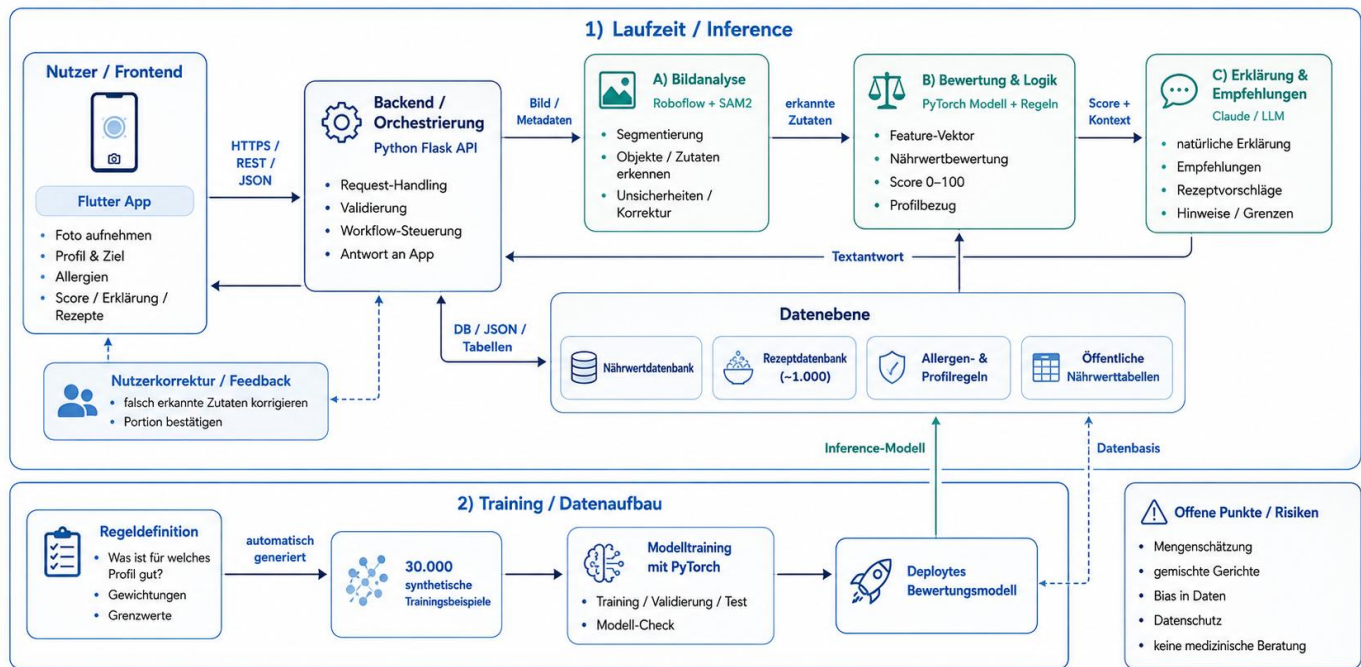


Abbildung 1: Funktions- und Schnittstellenübersicht von ExplainEat. Das Frontend ist funktional; der technische Schwerpunkt liegt auf Datenaufbau, Modelltraining, Inferenz und Integration.

Wichtig für die Ergebnisqualität: Ein Fehler in einem frühen Schritt – zum Beispiel eine falsche Zutat oder Portion – kann alle folgenden Berechnungen beeinflussen. Deshalb enthält der Ablauf einen verpflichtenden Korrekturschritt durch den Nutzer und eine separate Plausibilitätsprüfung vor der Erklärung.

4.1 Schritt 1: Profil und Ziel erfassen

- Ich erfasse nur Werte, die für die Bewertung benötigt werden, zum Beispiel Ziel, Aktivitätsniveau, Gewicht, Allergien oder Ernährungspräferenzen.
- Ich prüfe Wertebereiche auf Plausibilität und verhindere offensichtlich unrealistische Eingaben.
- Medizinisch sensible Angaben werden nicht wie gewöhnliche Fitnessziele behandelt.
- Kategorien werden strukturiert gespeichert, damit sie später eindeutig codiert werden können.

4.2 Schritt 2: Mahlzeit aufnehmen und ergänzen

- Der Nutzer nimmt ein Foto auf oder lädt ein Bild hoch.
- Zusätzliche Angaben wie Mahlzeitentyp, Sauce, Öl, Zubereitung, Verpackungswerte oder Portion können ergänzt werden.
- Die App weist darauf hin, dass ein Foto Salz, Zucker, versteckte Zutaten und Portionsgrösse nicht zuverlässig zeigt.
- Bild und strukturierte Metadaten werden gemeinsam an das Backend übertragen.

4.3 Schritt 3: Anfrage im Flask-Backend orchestrieren

- Das Backend nimmt Bild- und JSON-Daten über eine REST-Schnittstelle entgegen.
- Dateiformat, Pflichtfelder, Wertebereiche und technische Fehler werden validiert.
- Das Backend steuert die Reihenfolge von Bildanalyse, Nutzerbestätigung, Datenanreicherung, Modellaufruf und Erklärung.
- API-Schlüssel bleiben im Backend und werden nicht in der App gespeichert.
- Alle Teilergebnisse werden in einem kontrollierten Ergebnisobjekt zusammengeführt.

4.4 Schritt 4: Bildbereiche segmentieren und Lebensmittel erkennen

- SAM2 ist ein Segmentierungsmodell: Es kann Bildbereiche voneinander trennen, benennt Lebensmittel aber nicht automatisch zuverlässig.
- Für die Benennung ist zusätzlich eine Erkennungs- oder Klassifikationskomponente erforderlich. Die Dokumentation muss genau angeben, welches Modell dafür eingesetzt wird.
- Zu jedem Vorschlag werden Konfidenz und Modellversion gespeichert.
- Unsichere oder unbekannte Bestandteile werden sichtbar markiert und nicht als sichere Wahrheit behandelt.

4.5 Schritt 5: Nutzerbestätigung als Qualitätskontrolle

- Erkannte Zutaten werden vor der Bewertung angezeigt.
- Falsche Erkennungen können korrigiert, fehlende Zutaten ergänzt und Portionen bestätigt werden.
- Bestätigte Angaben haben Vorrang vor automatischen Schätzungen.
- Das Ergebnisobjekt unterscheidet gemessene, bestätigte und geschätzte Werte.

4.6 Schritt 6. Nährwerte und Kontext anreichern

- Bestätigte Zutaten werden Nährwertdaten zugeordnet.
- Einheiten werden vereinheitlicht und auf die bestätigte Portion umgerechnet.
- Zusätzliche Merkmale wie Gemüseanteil, Datenabdeckung oder Anteil unbekannter Zutaten werden berechnet.
- Fehlende Werte werden als fehlend markiert und nicht stillschweigend als Null interpretiert.

4.7 Schritt 7: Feature-Vektor erzeugen

- Alle Merkmale werden in eine feste, dokumentierte Reihenfolge gebracht.
- Kategorien werden mit Flags oder One-Hot-Encoding in Zahlen übersetzt.
- Numerische Werte werden mit Parametern skaliert, die ausschliesslich aus den Trainingsdaten berechnet wurden.
- Der resultierende Tensor hat den Datentyp float32 und die Form [Batchgrösse, Anzahl Merkmale].
- Training und Inferenz müssen exakt dasselbe Preprocessing verwenden.

4.8 Schritt 8: Score mit dem PyTorch-Modell berechnen

- Das Backend lädt Modellgewichte, Feature-Schema und Preprocessing-Artefakte gemeinsam.
- Das Modell wird in den Evaluationsmodus gesetzt und ohne Gradientenberechnung ausgeführt.
- Die Ausgabe wird auf erlaubte Wertebereiche begrenzt und auf technische Fehler geprüft.
- Allergien, harte Ausschlussregeln und medizinische Sicherheitsregeln bleiben ausserhalb des neuronalen Netzes.

4.9 Schritt 9: Plausibilität und Sicherheit prüfen

- Das System prüft, ob Portion, Zutaten und Datenabdeckung für eine belastbare Aussage ausreichen.
- Bei hoher Unsicherheit wird keine definitive Bewertung ausgegeben.
- Score, Warnhinweise und harte Regeln bleiben getrennte Ergebnisbestandteile.
- Die App muss ausdrücklich sagen können: „Eine zuverlässige Bewertung ist mit diesen Daten nicht möglich.“

4.10 Schritt 10: Erklärung mit LLM (Claude Code) erzeugen

- Claude erhält ein strukturiertes, bereits geprüftes Ergebnisobjekt und keine Verantwortung für die eigentliche Bewertung.
- Das Sprachmodell formuliert Score, Teilwerte, Stärken, offene Daten und Grenzen in verständliche Sprache um.

- Die Ausgabe wird auf Orientierung, nachvollziehbare Verbesserungen und passende Rezeptvorschläge begrenzt.
- Diagnosen, Therapie-, Medikamenten- oder Dosierungsempfehlungen sind ausgeschlossen.

4.11 Schritt 11: Ergebnis in der App anzeigen

- Die App zeigt bestätigte Zutaten, Datenqualität, Score, Teilbewertungen, Erklärung und mögliche Alternativen.
- Unsicherheit wird sichtbar dargestellt, damit keine Scheingenauigkeit entsteht.
- Der Prototypstatus und die fehlende medizinische Validierung werden klar gekennzeichnet.
- Der Nutzer kann Angaben korrigieren und die Bewertung erneut ausführen.

4.12 Schritt 12: Feedback kontrolliert weiterverwenden

- Korrekturen können als mögliche neue Datenpunkte gespeichert werden.
- Feedback wird nicht ungeprüft in das Training übernommen.
- Neue Daten werden bereinigt, versioniert und gegen feste Testfälle geprüft.
- Eine neue Modellversion wird nur übernommen, wenn die Verbesserung mit definierten Metriken messbar ist.

5. Der eigene KI-Kern: PyTorch-Modell und Feature-Vektor

5.1 Warum PyTorch im Mittelpunkt steht

PyTorch ist das Framework, mit dem ich das neuronale Netz definiere, trainiere, speichere und ausführe. Tensoren stellen numerische Daten dar, ein DataLoader liefert Trainingsbeispiele in Batches, das nn.Module beschreibt das Netz, Autograd berechnet die Gradienten und ein Optimizer passt die Gewichte an. Meine Eigenleistung liegt nicht in diesen Standardbausteinen, sondern in ihrer Anbindung und fachlich begründeten Kombination für ExplainEat.

- Für die Challenge zeigt PyTorch einen vollständigen Machine-Learning-Prozess und nicht nur den Aufruf einer fertigen KI-API.
- Für das Lernen kann ich Vorwärtslauf, Fehlerfunktion, Backpropagation, Optimierung und Inferenz praktisch erklären.
- Für den Ausbau kann ich reale Daten, zusätzliche Ausgaben und Unsicherheitsmodelle ergänzen.
- Der Nachteil ist die geringere Transparenz gegenüber einer direkten Regelberechnung. Deshalb sind Baseline-Vergleich, Tests und Versionierung notwendig.

5.2 Lernaufgabe und Beziehung zur Regel-Engine

Die aktuellen Zielwerte werden aus einer von mir definierten Regel-Engine erzeugt. Das neuronale Netz lernt somit zunächst, diese Regeln aus dem Feature-Vektor möglichst gut nachzubilden. Technisch ist das ein Surrogat- oder Approximationsmodell: Es ersetzt die Regel-Engine nicht automatisch durch „bessere Intelligenz“, sondern versucht, deren Ergebnis zu reproduzieren.

Kritische Einordnung: Ein sehr gutes Ergebnis auf synthetischen Daten beweist vor allem, dass das Modell die selbst erzeugten Regeln gelernt hat. Es beweist noch nicht, dass die Bewertung auf echten Mahlzeiten ernährungswissenschaftlich korrekt oder für neue Bevölkerungsgruppen verlässlich ist.

Der fachliche Mehrwert des Modells muss deshalb gegen eine direkte Regelberechnung geprüft werden. Mögliche Vorteile wären eine glattere Bewertung, bessere Generalisierung bei leicht veränderten Eingaben oder die spätere Nutzung fachlich bewerteter realer Daten. Falls kein messbarer Vorteil entsteht, ist die Regel-Engine die transparentere Baseline und das PyTorch-Modell bleibt ein Lern- und Integrationsnachweis.

5.3 Feature-Vektor: verständliche Definition

Der Feature-Vektor ist eine geordnete Liste von Zahlen, die eine Mahlzeit und den relevanten Kontext für das Modell beschreibt. Jede Position hat eine feste Bedeutung. Beispiel: Position 1 kann die Energie, Position 2 das Protein und eine spätere Position das Aktivitätsniveau enthalten. Die Reihenfolge darf zwischen Training und Inferenz nie abweichen.

- Numerische Merkmale: zum Beispiel Energie, Protein, Kohlenhydrate, Fett, Ballaststoffe, Zucker, Salz, Portion und Gemüseanteil.
- Kategorische Merkmale: zum Beispiel Mahlzeitentyp oder Ziel; sie werden mit Flags oder One-Hot-Encoding in Zahlen übersetzt.
- Qualitätsmerkmale: Datenabdeckung, Anteil unbekannter Zutaten, Konfidenz oder Kennzeichnung einer Nutzerbestätigung.
- Sicherheitsmerkmale: Allergien und harte medizinische Ausschlüsse sollten nicht nur im Modell liegen, sondern separat regelbasiert geprüft werden.

5.4 Technische Bedeutung des aktuellen KI-Kerns

Der aktuelle KI-Kern zeigt, dass ich einen vollständigen ML-Lebenszyklus technisch umsetzen kann. Seine wissenschaftliche Aussagekraft ist jedoch durch synthetische Zielwerte begrenzt. Der nächste qualitative Schritt besteht nicht in einem grösseren Netz, sondern in besseren, realen und fachlich geprüften Daten sowie in einem klaren Vergleich mit einfachen Baselines. Siehe kritische Reflexion.

6. Training, Evaluation und Ausbau des Modells

6.1 Datengenerierung

Aus den definierten Regeln erzeuge ich rund 30'000 synthetische Kombinationen aus Mahlzeitenmerkmalen, Profilmerkmalen und Zielscore. Der Vorteil ist, dass ich viele kontrollierte Fälle erzeugen und seltene Kombinationen gezielt abdecken kann. Der Nachteil ist, dass die Daten nur die Annahmen und Grenzen meiner Regeln enthalten. Fehler oder Verzerrungen im Regelwerk werden dadurch ebenfalls gelernt.

- Jeder Datensatz muss eine Versionsnummer und einen dokumentierten Generatorstand besitzen.
- Zulässige Wertebereiche und Verteilungen müssen beschrieben werden.
- Unrealistische Kombinationen sollen vermieden oder ausdrücklich als Testfälle markiert werden.
- Trainingsdaten dürfen keine Informationen aus Validierungs- oder Testdaten übernehmen.

6.2 Datenaufteilung und Preprocessing

1. Die Daten werden reproduzierbar in Training, Validierung und Test aufgeteilt.
2. Der Testanteil bleibt bis zur finalen Bewertung unberührt.
3. Mittelwerte, Standardabweichungen oder andere Skalierungsparameter werden ausschliesslich auf dem Trainingsanteil berechnet.
4. Kategorien und Feature-Reihenfolge werden in einem versionierten Schema gespeichert.
5. Das Backend verwendet exakt dieselben Transformationen wie das Trainingskript.

6.3 Trainingsablauf

1. Regeln, Feature-Schema und Zieldefinition festlegen.
2. Synthetische Trainingsbeispiele erzeugen und auf Plausibilität prüfen.
3. Trainings-, Validierungs- und Testdaten reproduzierbar trennen.
4. Modell trainieren und den Validierungsfehler beobachten.
5. Besten Modellzustand speichern; nicht nur den letzten Trainingsstand.
6. Modell, Feature-Schema, Skalierung, Konfiguration und Metriken gemeinsam versionieren.
7. Finale Bewertung auf dem unabhängigen Testanteil durchführen.
8. Ergebnis mit der direkten Regel-Engine und einfachen Baselines vergleichen.

6.4 Ausbaupfad der KI

- Stufe 1 – Reproduzierbarkeit: Seeds, Versionen, Feature-Schema, Split, Modellhash und Beispielinputs fixieren.
- Stufe 2 – Reale Testbilder: kleiner, manuell geprüfter Bilddatensatz mit einfachen und gemischten Mahlzeiten.
- Stufe 3 – Bestätigte Portionen: automatische Schätzungen durch Nutzerbestätigung ergänzen und Restunsicherheit dokumentieren.
- Stufe 4 – Fachlich bewertete Labels: Zielwerte durch qualifizierte Fachpersonen prüfen lassen, statt sie nur aus eigenen Regeln abzuleiten.
- Stufe 5 – Multi-Output-Modell: Gesamtscore, Teilwerte und Unsicherheit getrennt ausgeben.
- Stufe 6 – Active Learning: Fälle priorisieren, bei denen Modell, Regel-Engine und Nutzerkorrektur stark abweichen.
- Stufe 7 – Bias- und Subgruppenprüfung: Esskulturen, Gerichtstypen, Länder, Profile und Datenqualitätsstufen getrennt testen.
- Stufe 8 – Monitoring: Datensatz-, Feature-, Modell- und Metrikversionen mit bekannten Grenzen verknüpfen.

7. Zwei Use Cases und ihr Nutzen

7.1 Use Case 1 – gesunder Kraft- und Fitnesssportler

Der erste Use Case zeigt den vorgesehenen Kernnutzen in einem nicht-medizinischen Szenario. Ein gesunder Erwachsener möchte seine Mahlzeit im Zusammenhang mit Krafttraining und allgemeiner Fitness verstehen.

Aspekt	Beispiel
Profil	27 Jahre, gesund, vier Trainingseinheiten pro Woche, Ziel Muskelaufbau und allgemeine Fitness.
Mahlzeit	Poulet, Reis, Brokkoli und Sauce.
Bildanalyse	Lebensmittel werden als Vorschläge erkannt; die Sauce erhält eine geringere Konfidenz.
Nutzerkorrektur	Portionen und Sauce werden manuell bestätigt.
Feature-Vektor	Zum Beispiel Energie, Protein, Kohlenhydrate, Gemüseanteil, Portion, Aktivität, Ziel und Datenqualität.
PyTorch-Ausgabe	Profilbezogener Gesamtscore und – falls implementiert – Teilwerte.
Claude-Ausgabe	Verständliche Erklärung, warum bestätigte Proteinquelle und Gemüseanteil passen und wo Unsicherheit bleibt.

Nutzen dieses Profils

- Die Bildanalyse erzeugt eine erste Zutatenliste und reduziert den manuellen Aufwand.
- Die Bewertung berücksichtigt nicht nur Kalorien, sondern auch Ziel, Portion und Datenqualität.
- Teilwerte und Gründe sind verständlicher als eine einzelne Zahl ohne Kontext.
- Der Nutzer erkennt wiederkehrende Muster und kann Änderungen vergleichen.
- Der Score garantiert weder Muskelaufbau noch einen individuellen Leistungsfortschritt.

Beispielausgabe: Die Mahlzeit passt grundsätzlich gut zum hinterlegten Trainingsziel. Die bestätigte Proteinquelle und der Gemüseanteil sprechen für eine ausgewogene Hauptmahlzeit. Die Sauce konnte nur teilweise bewertet werden; deshalb ist die Datenqualität mittel.

7.2 Use Case 2 – medizinisch sensibles Profil

Dieser Use Case gehört nicht zum medizinisch validierten Kern des Prototyps. Er zeigt lediglich, wie ExplainEat in einer späteren, fachlich geprüften Ausbaustufe mehrere bestätigte Datenpunkte transparent zusammenführen könnte. Individuelle Grenzwerte dürfen nicht von der App erfunden werden.

Aspekt	Beispiel
Profil	58 Jahre, Diabetes und erhöhter Blutdruck. Zielbereiche stammen ausschliesslich von einer qualifizierten Fachperson.
Mahlzeit	Fertigsuppe, Weissbrot, Wurst und gesüsstes Getränk.
Bildanalyse	Gerichtsklassen werden vorgeschlagen; genaue Produktwerte sind aus dem Foto nicht zuverlässig ableitbar.
Zusatzdaten	Etikett- oder Barcode-Daten ergänzen Kohlenhydrate, Zucker, Salz/Natrium und Portion.
Regelprüfung	Datenabdeckung, Portion, verarbeitete Produkte und hinterlegte fachliche Grenzen werden geprüft.
PyTorch-Rolle	Nur ein fachlich validiertes Profilmodell dürfte eine vorsichtige Passungsbewertung liefern.
Claude-Rolle	Neutral erklären, welche bestätigten Merkmale beitragen, und welche Daten fehlen; keine Therapie.

Nutzen dieses Ausbauprofils

- Produktetiketten und Bildinformationen können gemeinsam dargestellt werden.
- Die App kann erklären, warum eine Bewertung unsicher ist und welche Daten fehlen.
- Mehrere profilrelevante Merkmale werden in einer verständlichen Zusammenfassung verbunden.
- Eine sachliche Alternative aus der Rezeptdatenbank kann vorgeschlagen werden, ohne Lebensmittel pauschal zu verbieten.
- Das Ergebnis kann als Gesprächsgrundlage dienen, ersetzt aber keine fachliche Entscheidung.

Zwingende Sicherheitsgrenze: Keine Diagnose, keine Anpassung von Insulin oder Medikamenten, keine Garantie zur Blutzucker- oder Blutdruckwirkung und keine pauschale Freigabe „für Diabetiker geeignet“. Bei unvollständigen Daten muss ExplainEat die Bewertung begrenzen oder ablehnen.

7.3 Vergleich der Use Cases

Dimension	Gesunder Sportler	Medizinisch sensibles Ausbauprofil
Hauptziel	Orientierung für Fitness- und Trainingsziel.	Verständliche Zusammenführung bestätigter Profil- und Produktdaten.
Bedeutung des Fotos	Schneller Einstieg, danach Bestätigung.	Nur erster Hinweis; Etikett- und Fachwerte sind entscheidend.
Rolle des PyTorch-Modells	Profilbezogene Bewertung mit überschaubarem Risiko.	Nur nach fachlicher Validierung und strengeren Schutzregeln.
Rolle der Regeln	Allergien, Plausibilität, Datenqualität und Unsicherheit.	Zusätzlich medizinische Ausschlüsse und fachlich definierte Grenzwerte.
Rolle von Claude	Erklärung und allgemeine Vorschläge.	Vorsichtige Erklärung; keine Diagnose oder Therapie.
Hauptrisiko	Scheingenauigkeit und fehlender Tageskontext.	Gesundheitsschaden durch falsche oder zu definitive Aussagen.